```
#Exploratory data analysis
#Visualization Matplotlib, seaborn
#Statistical using python
#EDA (analyse the data, visualize, understand the patterns)
```

```
import pandas as pd
df= pd.read_csv('/content/amazon_products_sales_data_cleaned.csv')
df.head()
```

	product_title	product_rating	total_reviews	purchased_last_month	discounted_price	original_price	is_best_seller	is_sponsor
0	BOYA BOYALINK 2 Wireless Lavalier Microphone f...	4.6	375.0	300.0	89.68	159.00	No Badge	Sponsor
1	LISEN USB C to Lightning Cable, 240W 4 in 1 Ch...	4.3	2457.0	6000.0	9.99	15.99	No Badge	Sponsor
2	DJI Mic 2 (2 TX + 1 RX + Charging Case), Wirel...	4.6	3044.0	2000.0	314.00	349.00	No Badge	Sp
3	Apple AirPods Pro 2 Wireless Earbuds, Active N...	4.6	35882.0	10000.0	162.24	162.24	Best Seller	Orgar
4	Apple AirTag 4 Pack. Keep Track of and find Yo...	4.8	28988.0	10000.0	72.74	72.74	No Badge	Orgar

```
df.describe()
```

	product_rating	total_reviews	purchased_last_month	discounted_price	original_price	discount_percentage
count	41651.000000	41651.000000	32164.000000	40613.000000	40613.000000	40613.000000
mean	4.399431	3087.106000	1293.665278	243.227289	257.611107	6.547151
std	0.386997	13030.460133	6318.323574	473.351545	496.633495	12.744715
min	1.000000	1.000000	50.000000	2.160000	2.160000	0.000000
25%	4.200000	82.000000	100.000000	29.690000	32.990000	0.000000
50%	4.500000	343.000000	200.000000	84.990000	89.000000	0.000000
75%	4.700000	1886.000000	400.000000	224.000000	229.990000	8.490000
max	5.000000	865598.000000	100000.000000	5449.000000	5449.000000	85.420000

```
df.shape
```

```
(42675, 17)
```

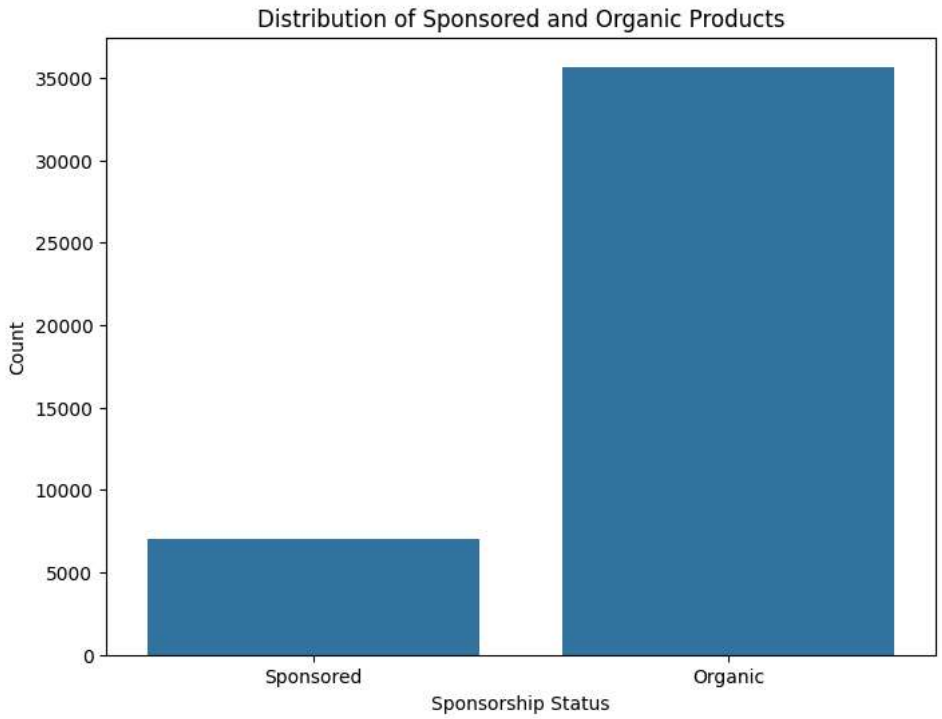
```
import pandas as pd
df= pd.read_csv('/content/amazon_products_sales_data_cleaned.csv')
df['is_sponsored'].value_counts()
```

is_sponsored	count
Organic	35664
Sponsored	7011

dtype: int64

```
import matplotlib.pyplot as plt
import seaborn as sns

plt.figure(figsize=(8, 6))
sns.countplot(data=df, x='is_sponsored')
plt.title('Distribution of Sponsored and Organic Products')
plt.xlabel('Sponsorship Status')
plt.ylabel('Count')
plt.show()
```




```
df.columns
```

```
Index(['product_title', 'product_rating', 'total_reviews',
       'purchased_last_month', 'discounted_price', 'original_price',
       'is_best_seller', 'is_sponsored', 'has_coupon', 'buy_box_availability',
       'delivery_date', 'sustainability_tags', 'product_image_url',
       'product_page_url', 'data_collected_at', 'product_category',
       'discount_percentage'],
      dtype='object')
```

```
df.dtypes
```

	0
product_title	object
product_rating	float64
total_reviews	float64

```
df.isnull().sum()
```

	0
original_price	float64
product_title	0
product_rating	1024
is_sponsored	object
total_reviews	1024
purchased_last_month	10511
buy_box_availability	object
discounted_price	2062
original_price	2062
sustainability_tags	object
is_best_seller	0
is_sponsored	0
product_page_url	object
has_coupon	0
buy_box_availability	14653
product_category	object
delivery_date	11983
sustainability_tags	39267

dtype: object

product_image_url	0
product_page_url	2069
data_collected_at	0
product_category	0
discount_percentage	2062

dtype: int64

```
df=df.dropna() #remove all null values from data sets
```

```
# Display descriptive statistics for the DataFrame
display(df.describe())
```

	product_rating	total_reviews	purchased_last_month	discounted_price	original_price	discount_percentage
count	1846.000000	1846.000000	1846.000000	1846.000000	1846.000000	1846.000000
mean	4.475244	9788.252979	1132.881907	100.591463	110.882763	12.316262
std	0.238853	35035.068471	4397.243474	153.530539	163.625290	14.201553
min	3.400000	15.000000	50.000000	4.950000	5.990000	0.000000
25%	4.400000	1033.250000	100.000000	22.990000	26.950000	0.000000
50%	4.600000	2528.000000	300.000000	31.050000	49.950000	6.730000
75%	4.600000	7235.000000	500.000000	79.950000	79.950000	25.010000
max	5.000000	865598.000000	100000.000000	1159.990000	1199.990000	63.150000

```
#statistics
df['total_reviews'] = df['total_reviews'].astype(str).str.replace(',', '', regex=False)
df['total_reviews'] = pd.to_numeric(df['total_reviews'])
df['total_reviews'].mean()
```

```
np.float64(9788.25297941495)
```

```
df['total_reviews'].median()
```

```
2528.0
```

```
df['total_reviews'].mode()
```

	total_reviews
0	7235.0

dtype: float64

Task

Perform correlation analysis and grouped analysis on the dataframe `df`.

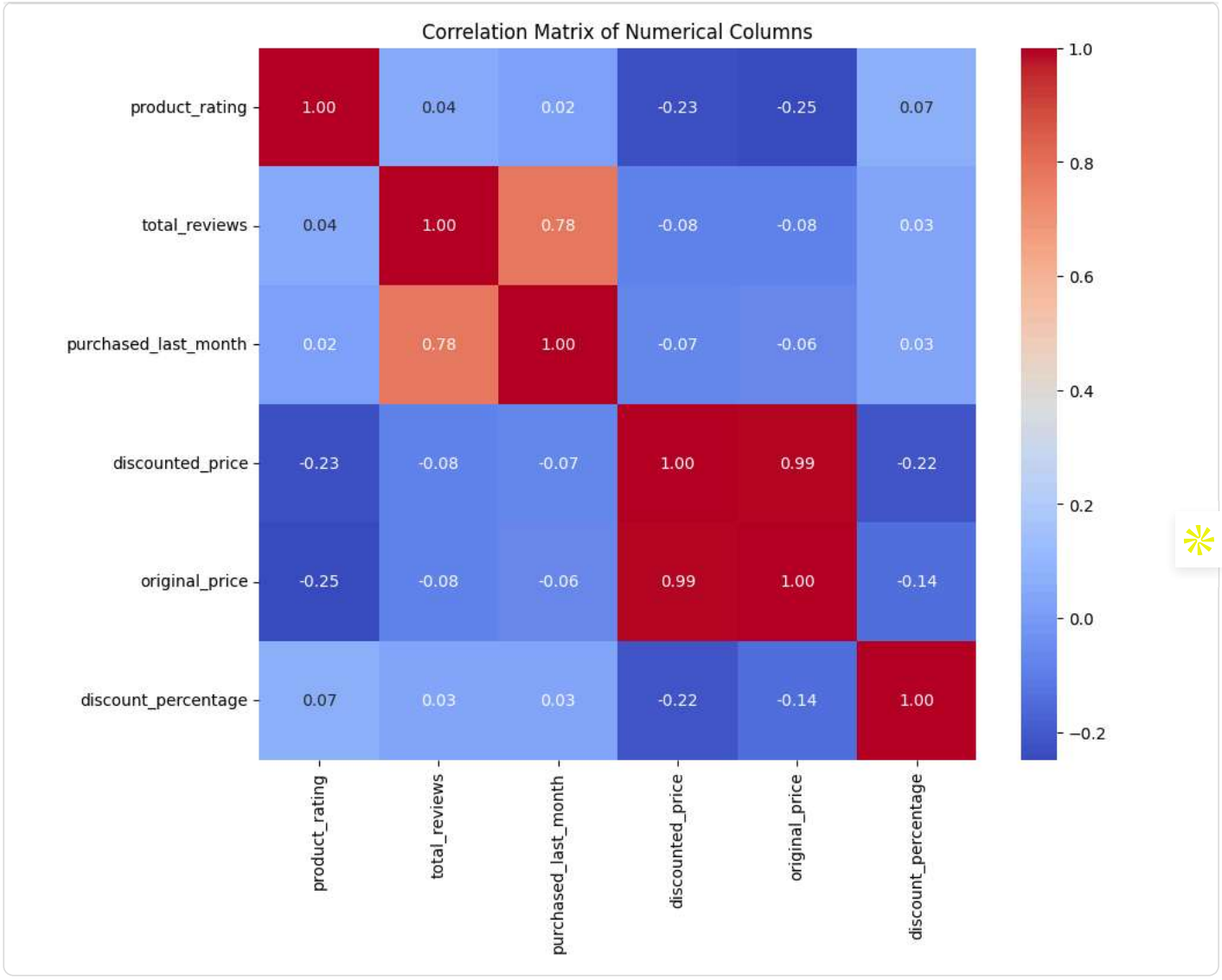
Correlation analysis

Subtask:

Calculate and visualize the correlation matrix for the numerical columns in the DataFrame.

Reasoning: Select numerical columns, calculate the correlation matrix, and visualize it using a heatmap.

```
numerical_cols = df.select_dtypes(include=['float64', 'int64'])
correlation_matrix = numerical_cols.corr()
plt.figure(figsize=(10, 8))
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm', fmt=".2f")
plt.title('Correlation Matrix of Numerical Columns')
plt.show()
```



Grouped analysis

Subtask:

Choose a categorical column and perform a grouped analysis on a numerical column to compare statistics across different groups.

Reasoning: Perform grouped analysis on the selected categorical and numerical columns.

```
grouped_analysis = df.groupby('product_category')['purchased_last_month'].agg(['mean', 'median', 'count'])
display(grouped_analysis)
```

product_category	mean	median	count
Cameras	488.551402	500.0	428
Chargers & Cables	1340.000000	300.0	10
Gaming	1642.857143	400.0	7
Headphones	1166.666667	450.0	12
Laptops	1914.402810	500.0	427
Networking	2296.428571	1000.0	14
Other Electronics	562.837838	300.0	148
Phones	1318.821293	300.0	526
Power & Batteries	2123.076923	500.0	13
Printers & Scanners	957.142857	50.0	105
Smart Home	442.857143	100.0	119
Speakers	260.000000	200.0	5
Storage	1000.000000	1000.0	1
TV & Display	903.571429	200.0	28
Wearables	2766.666667	2000.0	3

Next steps: Generate code with `grouped_analysis` New interactive sheet

Summary:

Data Analysis Key Findings

- The correlation matrix for numerical columns was calculated and visualized. Specific correlation values can be observed in the generated heatmap (not shown).
- A grouped analysis on 'purchased_last_month' by 'product_category' was performed. This analysis provided the mean, median, and count of 'purchased_last_month' for each product category, stored in the `grouped_analysis` DataFrame.

Insights or Next Steps

- Investigate product categories with higher or lower purchase rates based on the grouped analysis results to identify potential trends or areas for improvement.
- Analyze the correlations between numerical features to understand relationships and inform potential feature selection or engineering for future modeling.

Task

Perform correlation analysis, grouped analysis, and statistical tests on the data.

Statistical testing

Subtask:

Perform relevant statistical tests on the data.

Reasoning: Choose a statistical test to compare the mean product rating between sponsored and organic products. This is a common scenario where a t-test is appropriate.

```
from scipy import stats

sponsored_ratings = df[df['is_sponsored'] == 'Sponsored']['product_rating'].dropna()
organic_ratings = df[df['is_sponsored'] == 'Organic']['product_rating'].dropna()

t_statistic, p_value = stats.ttest_ind(sponsored_ratings, organic_ratings)

print(f"T-statistic: {t_statistic}")
print(f"P-value: {p_value}")

alpha = 0.05
if p_value < alpha:
    print("Reject the null hypothesis: There is a significant difference in mean product ratings between sponsored and organic prc
else:
```